

X-39MATRIX · LAYER 10 · SPRINT 2 DESIGN SPEC

Migración del AIR de SHA-256 a Rescue-Prime

Versión: v1.0-design Fecha: 2026-06-26 Autor: Jose Luis Olivares Esteban · operador soberano PGP: C3E062EB251A11851C0B4FFD06870F0655D5BBE8 Estado: DISEÑO · pre-implementación · pendiente financiación (NLnet/OpenSats/DFINITY)

0. Honestidad cypherpunk previa

Este documento describe el diseño del Sprint 2 de Layer 10. No hay código Rust de layer10-prove / layer10-verify en el repo aún. El Sprint 1 (Q3 2026) creará el harness Winterfell con AIR SHA-256 como baseline mensurable. Este Sprint 2 (Q4 2026 / Q1 2027) reemplazará el AIR por Rescue-Prime para conseguir el orden-de-magnitud de eficiencia. La implementación arrancará cuando exista financiación confirmada.

1. Problema a resolver

El AIR ("Algebraic Intermediate Representation") es la "circuitaria" de un zk-STARK: las restricciones algebraicas que el prover debe satisfacer. La función hash usada dentro de ese circuito es el cuello de botella del 80–90 % del tiempo de proving.

SHA-256 en AIR: - Operaciones bitwise (XOR/AND/SHL) → costoso reducir a polinomios de grado bajo sobre un campo grande. - ~27.000 restricciones para una sola compresión SHA-256 (Winterfell 2026). - Prover time típico: ~3,0 s por hash en circuito (Polygon Zero benchmarks, ene 2026).

Rescue-Prime en AIR: - Diseñado para ser "zk-friendly": permutación Rescue-XLIX usando potencias x^d y $x^{(1/d)}$ que producen restricciones de grado bajo nativamente. - ~100–200 restricciones por hash → ~140x más eficiente en constraint count. - Prover time: ~0,6 s por hash en circuito → ~5x más rápido en prover total. - Coste nativo (CPU): ~5x más lento que SHA-256, pero irrelevante porque el cuello de botella es el AIR, no la CPU.

Conclusión cuantitativa (Winterfell 2026, benchmark Polygon Miden):

Métrica	SHA-256 (Sprint 1)	Rescue-Prime (Sprint 2)	Ratio
Constraints / hash	~27.000	~100–200	~140x mejor
Prover time / circuito	~3,0 s	~0,6 s	~5x mejor
Tamaño de prueba	~25 KB	~18 KB	~1,4x mejor
Verificación en navegador	~250 ms	~120 ms	~2x mejor
Seguridad cuántica	128 bits (vs Grover)	128 bits (vs Grover + algebraic AIR)	equivalente

2. Riesgos y trade-offs honestos

2.1 Riesgos técnicos

1. **Madurez criptanalítica:** Rescue-Prime es un primitivo joven (2020, Szepieniec et al.). SHA-256 tiene >20 años de análisis público. Mitigación: usar parámetros conservadores (rondas ≥ 7 , security margin $\geq 1,5\times$) y publicar el AIR para audit independiente vía NLnet/Cure53.
2. **No-NIST:** SHA-256 está en FIPS 180-4. Rescue-Prime no está estandarizado por NIST y probablemente no lo estará en este ciclo. Mitigación: documentar claramente este trade-off y mantener un modo "conservador" SHA-256 opcional para aplicaciones regulatorias (eIDAS 2.0, NIS2) donde se exija FIPS.
3. **Compatibilidad con anclajes BTC existentes:** Los `.ots` actuales del corpus están anclados con SHA-256 (estándar Bitcoin). Layer 10 NO afecta a esto; sólo cambia el hash interno del circuito zk-STARK. El anclaje BTC sigue siendo SHA-256.

2.2 Trade-off de adopción

- Si elegimos Rescue-Prime puro → mejor performance pero menos auditores familiarizados con el primitivo.
 - Si elegimos modo dual (SHA-256 conservative + Rescue-Prime performance) → más código a mantener pero máxima flexibilidad. **Decisión preferida: modo dual con flag de compilación.**
-

3. Plan de implementación (Sprint 2)

3.1 Pre-requisitos (deben estar resueltos antes)

- [] Sprint 1 completado: harness Winterfell + AIR SHA-256 baseline en `layer10-prove/src/air_sha256.rs`.
- [] Tests E2E del Sprint 1 reproducibles en CI público (GitHub Actions).
- [] Auditor externo confirmado (Cure53 / Trail of Bits / SBA Research) vía financiación NLnet NGIO Security Audit Fund.

3.2 Tareas Sprint 2

Hito 1 (semanas 1–3): Implementar `layer10-prove/src/air_rescue_prime.rs` usando `winter-math` + `winterfell` 0.13. Parámetros: - Campo: `BaseElement` = primo de 64 bits (compatible con Winterfell). - Tamaño de estado: 8 elementos (256-bit security). - Rondas: 7 (security margin $1,5\times$ sobre el mínimo teórico). - α (power map): 7 (estándar para campos pequeños).

Hito 2 (semanas 4–5): Crear `air_dual.rs` que permita seleccionar SHA-256 vs Rescue-Prime vía feature flag de Cargo (`features = ["rescue", "sha256-conservative"]`).

Hito 3 (semanas 6–7): Benchmark E2E sobre el corpus público (mismo input que Sprint 1): - Generar 1.000 pruebas con cada AIR. - Medir: prover time, proof size, verify time, RAM peak. - Publicar tabla comparativa firmada PGP + anclada en BTC.

Hito 4 (semana 8): Documentar el RFC v2.0 con la decisión final y los benchmarks. Anclar en BTC. Crear PR en repo público con etiqueta `layer10-sprint2`.

3.3 Criterios de "Done" verificables

- [] CI público pasa el harness completo ($\geq N$ tests) en commit firmado PGP.
- [] Pruebas E2E publican proof + verify keys + benchmarks como artefactos descargables en HTTPS.
- [] Audit externo (financiado) emite reporte público con hallazgos.
- [] Whitepaper v2.0 anclado en bloque BTC mainnet (vía OTS).

Importante: No declararemos el Sprint 2 "done" hasta que estos 4 criterios estén satisfechos y verificables por cualquier auditor en < 5 min.

4. Dependencias externas (mínimas, todas FOSS)

- `winterfell` = "0.13" — Polygon Miden / Novi (MIT)
- `winter-math` = "0.13" — campo finito y polinomios (MIT)
- `winter-utils` = "0.13" — utilidades de serialización (MIT)
- (Opcional) `plonky3` = "0.1" — target de migración futura para producción escala

No se introducirán dependencias propietarias, con telemetría o no auditables.

5. Coste estimado y plan de financiación

Concepto	Coste estimado
2 ingenieros Rust senior × 8 semanas	€ 32.000
Audit externo Cure53 (Layer 10 spec + Sprint 2)	€ 40.000
Infraestructura CI + benchmark hardware	€ 3.000
Documentación y publicación pública	€ 5.000
Total Sprint 2	€ 80.000

Plan de financiación (sin overclaim): - **NLnet NGIO PET:** € 50.000 (target) — diseño + implementación - **NLnet NGIO Security Audit Fund:** € 30.000 (target) — audit Cure53 - **OpenSats General Fund:** \$30.000–50.000 (target) — implementación open source con impacto sobre Bitcoin (anclaje OTS) - **DFINITY Post-Quantum RFP:** \$25.000 (target tier intermedio) — adaptación a canisters ICP

Estado: todos los grants están en fase de borrador/draft. **No hay financiación confirmada al 26-06-2026.**

6. Referencias técnicas

1. Szepieniec et al., "Rescue-Prime: a Standard Specification (SoK)", 2020. <https://eprint.iacr.org/2020/1143>
 2. Anatomy of a STARK (chapter on Rescue-Prime). <https://aszepieniec.github.io/stark-anatomy/rescue-prime.html>
 3. Polygon Miden Winterfell repo. <https://github.com/facebook/winterfell>
 4. KU Leuven COSIC — Rescue-Prime official site. <https://www.esat.kuleuven.be/cosic/sites/rescue/about-rescue-prime/>
 5. zk-friendly hash functions comparison. <https://www.zellic.io/blog/zk-friendly-hash-functions>
-

7. Próximos pasos (en orden estricto)

1. Cerrar Sprint 1 (AIR SHA-256 baseline) — Q3 2026.
2. Obtener financiación NLnet NGIO PET → activar Sprint 2.
3. Audit externo confirmado → activar Hito 1.
4. Implementar Hitos 1–4.

5. Publicar RFC v2.0 + benchmarks anclados en BTC.

6. Migración a Plonky3 → Sprint 3 (cuando haya recursos para producción escala).

No confíes. Verifica. Si los benchmarks que prometemos no se cumplen, este documento queda anclado en Bitcoin como prueba pública de la falla. Cypherpunk Honesty: assumed.